

High-Level Design (HLD)

Professional-Skepticism Engine — Multi-Agent Audit Investigation Crew

Field	Value	Field	Value
Project	Professional-Skepticism Engine	Document	High-Level Design (HLD)
Version	1.1	Status	Reviewed — aligned with PRD v1.1
Owner	Capstone Team 01 (2 members)	Date	2026-06-25
Pattern	Multi-agent (debate + verify)	Stack	LangGraph · Claude · Streamlit · SQLite

1. Introduction

This HLD describes the system-level design of the Professional-Skepticism Engine: its architecture, major components, data flow, technology choices, integrations, and cross-cutting concerns. It is the bridge between the PRD (what & why) and the LLD (how, in detail).

1.1 Scope

In scope: the multi-agent investigation pipeline, grounding services (RAG + external APIs), human-in-the-loop, persistence, and the reporting/audit layer for the 5-day MVP. Out of scope: production ERP connectors, multi-tenant infra, autonomous ledger posting.

2. System Overview

For each flagged transaction (a “case”), a crew of role-specialised Claude agents assembles evidence, runs an adversarial Challenger/Defender debate, adjudicates a risk verdict, verifies it is grounded, and — for uncertain or material cases — pauses for a human. State and the debate transcript are persisted; every decision is audited.

3. Architecture

3.1 Architectural style

- **Orchestrated multi-agent graph** — LangGraph StateGraph owns control flow and durable state.
- **Role-separated agents** — each agent is an independent Claude call with one objective and a typed I/O contract.
- **Adversarial + verification loop** — independence of Challenger / Defender / Verifier is the core mechanism.
- **Grounding-before-judgement** — RAG + external APIs build evidence before any reasoning.
- **Human-in-the-loop via durable interrupt** — confidence-gated pause/resume.

3.2 Component view

Component	Responsibility	Key tech
Orchestrator	Graph control flow, routing, bounded re-runs, checkpointing	LangGraph StateGraph
Agent layer	6 role-specialised Claude agents (supervisor, evidence, challenger, defender, adjudicator, verifier)	langchain-anthropic
Grounding services	Policy RAG (vector DB) + external-API connectors with cache	Chroma/FAISS · httpx

Component	Responsibility	Key tech
State & persistence	Durable state + debate transcript + immutable audit log	SQLite checkpointer + tables
HITL / review	Confidence-gated interrupt; reviewer decision + signature	LangGraph interrupt/resume
Presentation	Case list, debate-transcript viewer, review queue, dashboard	Streamlit
Cross-cutting	Secrets, observability, eval harness	Key store · LangSmith · pytest

4. End-to-End Data Flow

#	Stage	Data in → out
1	Intake	flagged txn → case
2	Plan	case → plan + routing
3	Evidence	case → evidence{} (RAG + history + APIs) + citations
4	Debate	case+evidence → challenger_view, defender_view
5	Adjudicate	evidence+views → adjudication{risk·conf·disposition}
6	Verify	verification{passed·issues}; loop to adjudicator if failed (max 2 rounds → escalate)
7	Gate	routing: auto-clear / reviewer / partner review (≥USD 25,000, high-risk, or failed verification)
8	Human review	bundle → review{action·signature}
9	Report+Audit	all → case file + audit_history[]

5. Technology Stack

Layer	Technology
Orchestration	LangGraph (StateGraph + SQLite checkpointer, interrupt/resume)
LLM	Anthropic Claude — Sonnet for reasoning agents, Haiku for report generation
Schemas	Pydantic v2
RAG	Chroma / FAISS + embeddings (hybrid search + re-rank)
HTTP / connectors	httpx (timeout + cache + retry)
Persistence	SQLite (checkpoints, cases, reviews, audit_log)
UI	Streamlit
Quality / Ops	pytest + eval harness · Docker · LangSmith (optional)

6. Integration / External Interfaces

Interface	Direction	Purpose	Scope
Anthropic Claude	out	Every agent (debate/adjudicate/verify)	MVP
Frankfurter / exchangerate.host	out	Historical FX evidence	MVP
Google Shopping / Open Food Facts	out	Price-reference evidence	Future

Interface	Direction	Purpose	Scope
GST / GSTN verify	out	Vendor / tax validity	Future
Streamlit UI	in/out	Upload, review, resume	MVP

7. Deployment View

- **MVP:** single container (Docker) running the orchestrator + Streamlit, with a local SQLite file and a local Chroma index. One Anthropic API key from an environment secret.
- **Future:** split UI / worker services, managed Postgres checkpointer, a message queue for fan-out, and per-tenant isolation — the agent logic is unchanged.

```
[ User ]--HTTP-->[ Streamlit UI ]--calls-->[ Orchestrator (LangGraph) ]
                                     |--> Claude API (agents)
                                     |--> Vector DB (policy RAG)
                                     |--> External APIs (FX/price) [+cache]
                                     |--> SQLite (checkpoints, audit)
[ Reviewer ]--approve/reject-->[ UI ]--resume(thread_id)-->[ Orchestrator ]
```

8. Cross-Cutting Concerns

Concern	Approach
Reliability	Checkpointed state + transcript; bounded re-runs; API retry + cache fallback
Security	Keys in secret store; PII not logged in clear; least privilege; SoD on approvals
Observability	Per-agent traces (tokens/latency/cost) + run metrics; LangSmith optional
Scalability	Stateless agent calls; future queue + fan-out; per-tenant isolation
Cost	Rules pre-filter; debate-round caps; evidence cache; model routing per role
Auditability	Immutable hash-chained audit log incl. full debate transcript

9. Key Design Decisions

Decision	Rationale
Multi-agent debate (not single prompt)	Independent challenge/defence reduces false positives & bias; provable via A/B eval
Separate Verifier agent	A model can't reliably check itself; independent grounding gate catches hallucination
LangGraph + checkpointer	Durable, resumable human-in-the-loop and bounded re-run loops
RAG for policy	Company-specific, defensible verdicts; policy change = data change
Model routing per role	Cost/quality balance; deterministic temps for judge/verify, higher for advocates

10. Assumptions, Constraints & Risks

- **Assumptions:** synthetic/sample cases acceptable; small seeded policy KB; Anthropic key available.
- **Constraints:** 2 devs · 5 days; debate rounds capped; confidence model-reported (not calibrated).
- **Top risks:** latency/cost of debate (mitigate: caps + cache); agents losing independence (mitigate: strict role isolation); verifier misses (mitigate: citation ids + seeded tests).

Appendix A — Architecture Diagram

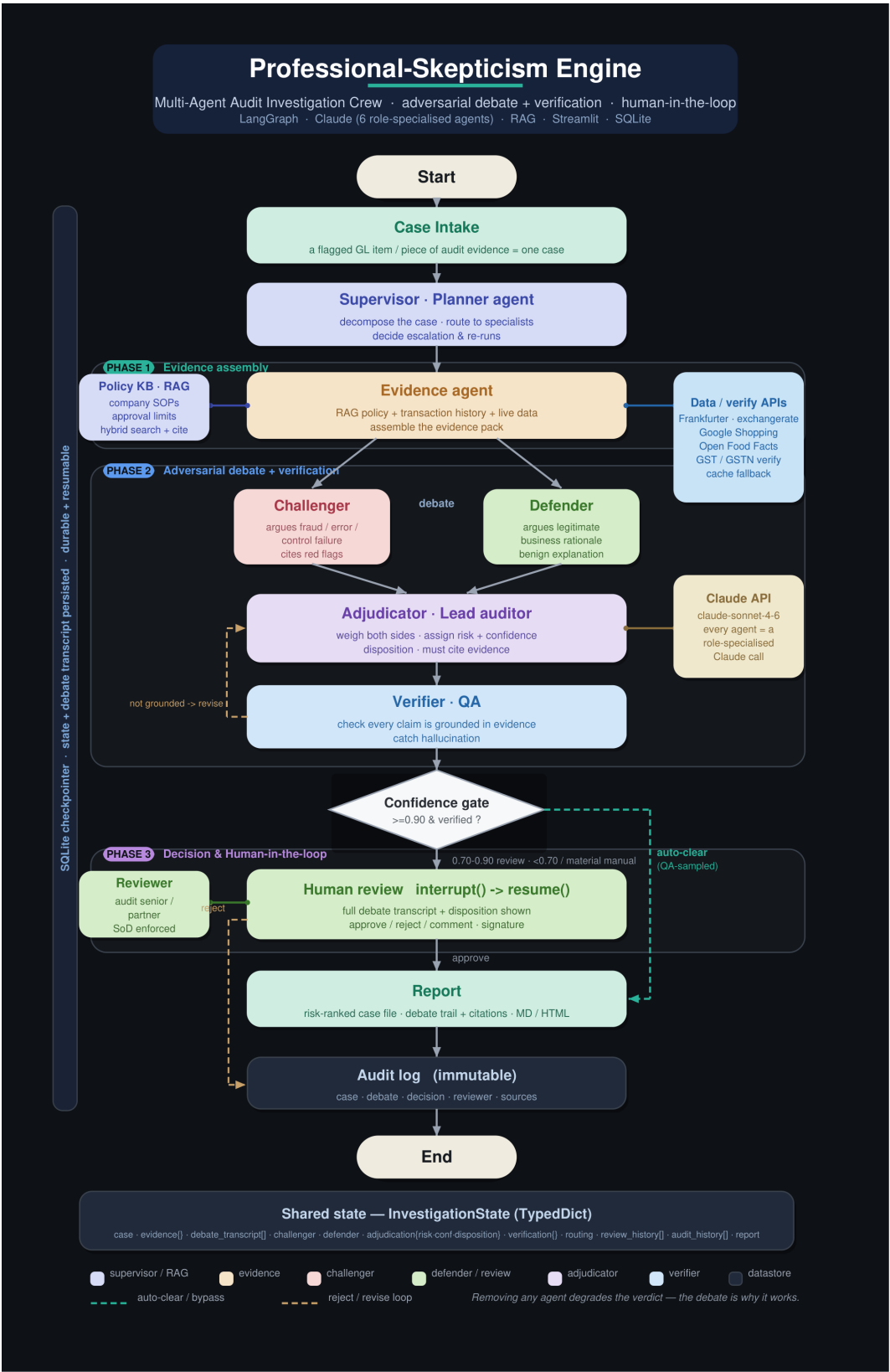


Figure 1 — System architecture (see PRD Appendix A for full detail).